7

Improving the Load Time Using Server-Side Rendering

After implementing authentication using JWTs, let's focus on optimizing the performance of our blog app. We are going to start by benchmarking the current load time of our application and learn about various metrics to consider. Then, we are going to learn how to render React components and fetch data on the server. At the end of this chapter, we are going to briefly cover advanced server-side rendering concepts.

In this chapter, we are going to cover the following main topics:

- Benchmarking the load time of our application
- Rendering React components on the server
- Server-side data fetching
- Advanced server-side rendering

Technical requirements

Before we start, please install all requirements mentioned in ***Chapter 1***, *Preparing For Full-Stack Development*, and ***Chapter 2***, *Getting to Know Node.js and MongoDB*.

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub: **https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch7**.

The CiA video for this chapter can be found at: **https://youtu.be/0OlmicibYWQ**

Benchmarking the load time of our application

Before we can get started improving the load time, we first must learn about the metrics to benchmark the performance of our application. The main metrics for measuring the performance of web applications are called **Core Web Vitals**, and they are as follows:

- **First Contentful Paint (FCP)**: This measures the loading performance of an app by reporting the time until the first image or text block is rendered on the page. A good target would be to get this metric below 1.8 seconds.
- **Largest Contentful Paint (LCP)**: This measures the loading performance of an app by reporting the time until the largest image or text block is visible within the viewport. A good target would be to get this metric below 2.5 seconds.
- **Total Blocking Time (TBT)**: This measures the interactivity of an app by reporting the time between the FCP and a user being able to interact with the page. A good target would be to get this metric below 200 milliseconds.
- **Cumulative Layout Shift (CLS)**: This measures the visual stability of an app by reporting unexpected movement on the page during loading, such as a link first being loaded on the top of the page, but then getting pushed further down to the bottom when other elements load. While this metric does not directly measure the actual performance of the app, it is still an important metric to consider, as it can lead to annoying the users when they attempt to click on something, but the layout shifts.

All these metrics can be measured by using the open-source **Lighthouse** tool, which is also available from the Google Chrome DevTools under the

Lighthouse panel. Let's get started benchmarking our app now:

1. Copy the **`ch6`** folder to a new **`ch7`** folder, as follows:

```
$ cp -R ch6 ch7
```

2. Open the **`ch7`** folder in VS Code, open a Terminal, and run the frontend with the following command:

```
$ npm run dev
```

3. Make sure the **`dbserver`** container is running in Docker.
4. Open a new Terminal and run the backend with the following command:

```
$ cd backend
$ npm run dev
```

5. Go to **`http://localhost:5173`** in Google Chrome and open the inspector (right-click and then press **Inspect**).

NOTE

It would be best to do this in an incognito tab so that extensions do not interfere with the measurements.

6. Open the **Lighthouse** tab (it might be hidden by the >> menu). It should look as follows:

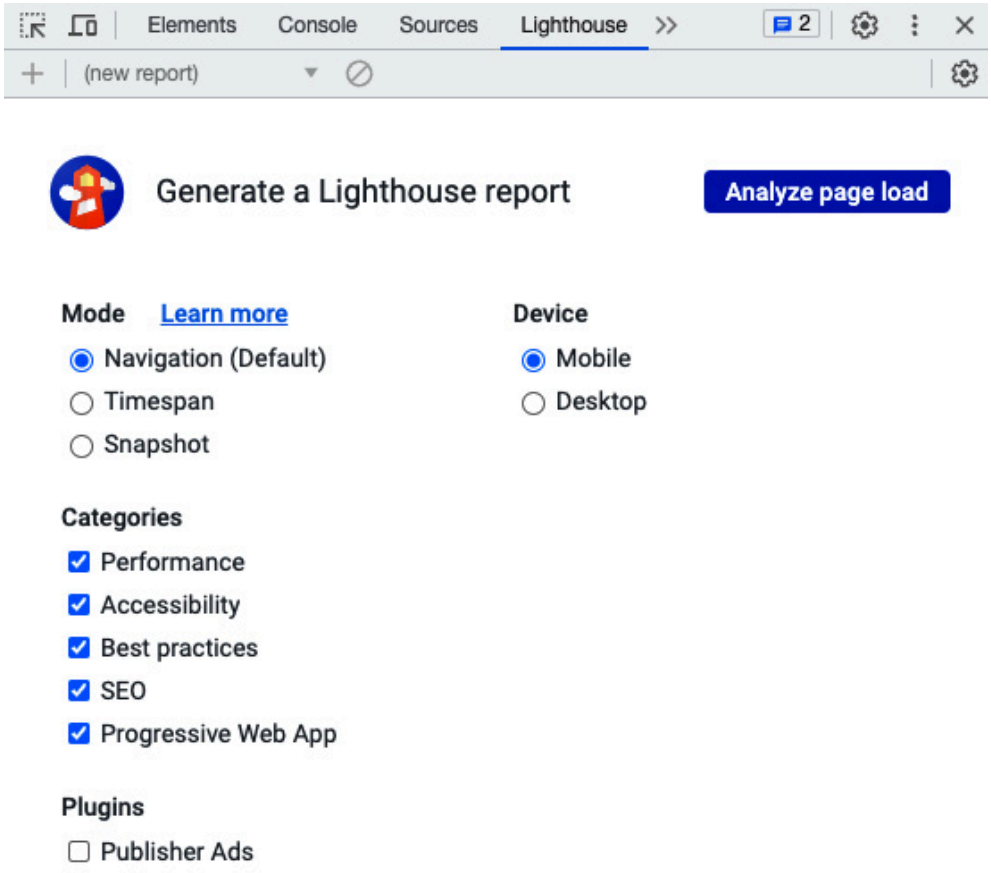

Figure 7.1 – The Lighthouse tab in Google Chrome DevTools

7. In the **Lighthouse** tab, leave all options as their default settings and click on the **Analyze page load** button.
   Lighthouse will start analyzing the website and give a report with metrics such as **First Contentful Paint**, **Largest Contentful Paint**, **Total Blocking Time**, and **Cumulative Layout Shift**. As we can see, our app already performs quite well in terms of TBT and CLS but performs particularly badly in terms of FCP and LCP. See the following screenshot for reference:

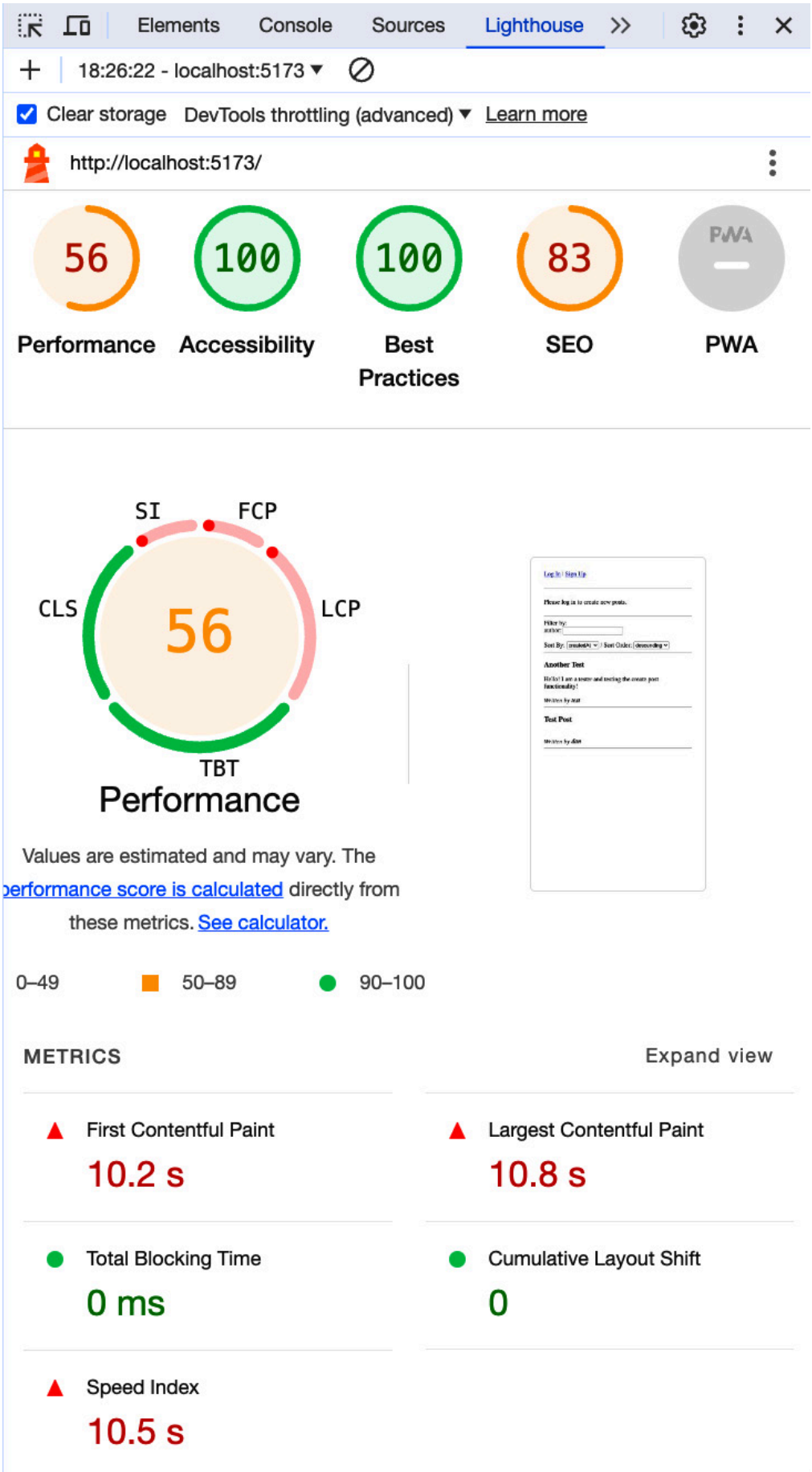

Figure 7.2 – Lighthouse results when analyzing our app in development mode (while hovering the cursor over the performance score)

There are two reasons why the paint takes so long. Firstly, we are running the server in dev mode, which generally makes everything slower. Additionally, we are rendering everything on the client side, which means that the browser first must download and execute our JavaScript code before it can start rendering the interface. Let's statically build our frontend and benchmark again now:

1. Install the **`serve`** tool globally with the following command, which is a tool that runs a simple web server:

```
$ npm install -g serve
```

2. Build the frontend with this command (execute it in the root of our project):

```
$ npm run build
```

3. Statically serve our app by running the following command:

```
$ serve dist/
```

4. Open **`http://localhost:3000`** in Google Chrome and run Lighthouse again (you may have to clear the old reports or click the list in the top left and select **(new report)** to analyze again).
   You should see the results of the new benchmark on the statically served frontend, which is closer to how it would be served in production. You can see an example of the results in the following screenshot:

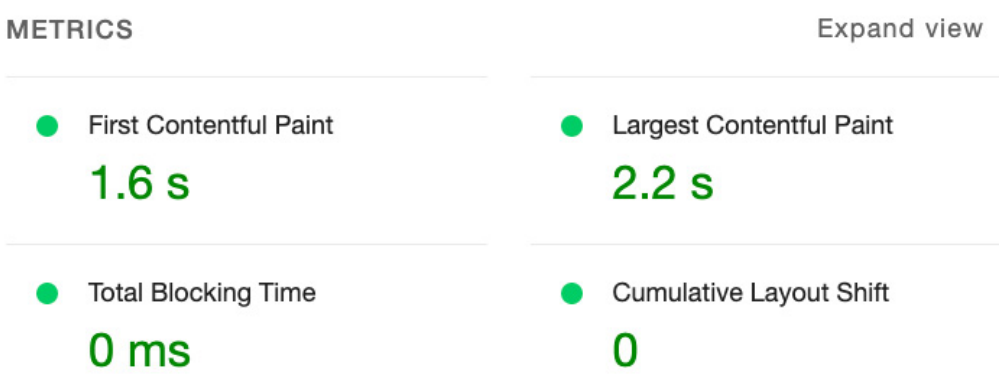

Figure 7.3 – Lighthouse report results on our statically built app

Now, the results are pretty good! However, it could still be improved further. Additionally, **Core Web Vitals** do not take into account the cascad-

ing requests to get the author usernames. While the first and largest contentful paints are fast in our app, the author names are not even loaded yet at that point. In addition to the Lighthouse report, we can also take a look at the **Network** tab to further debug the performance of our app, as follows:

1. In DevTools, go to the **Network** tab.
2. Refresh the page while the tab is open. You will see a waterfall diagram and the measured time to make requests, as shown in the following screenshot:

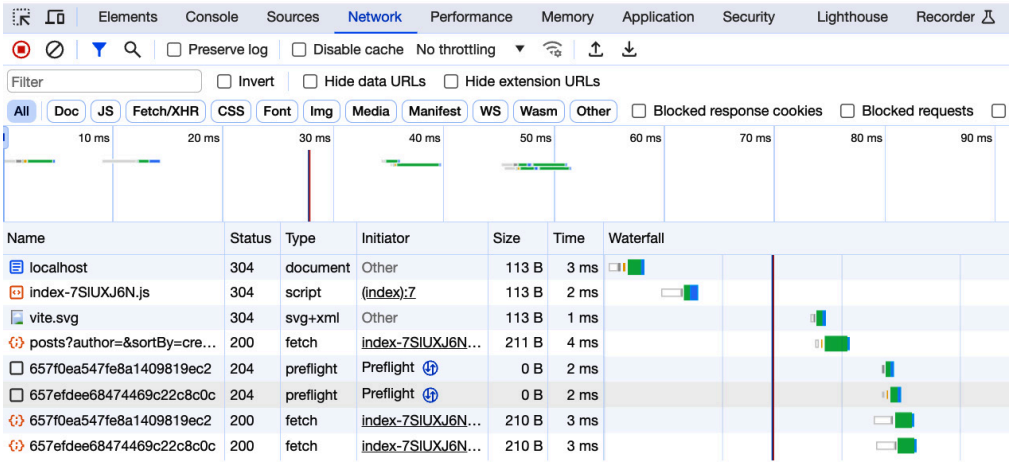

Figure 7.4 – The waterfall diagram on the Network tab

But the times are extremely low (all below 10 ms). This is because our backend is running locally, so there is no network delay. This is not a realistic scenario. In production, we would have latency on every request that we make, so we would first have to wait for the blog posts to be pulled, then fetch the names of authors for each author separately. We can use the DevTools to simulate a slower network connection; let's do that now:

1. At the top of the **Network** tab, click on the **No throttling** dropdown.
2. Select the **Slow 3G** preset. See the following screenshot for reference:

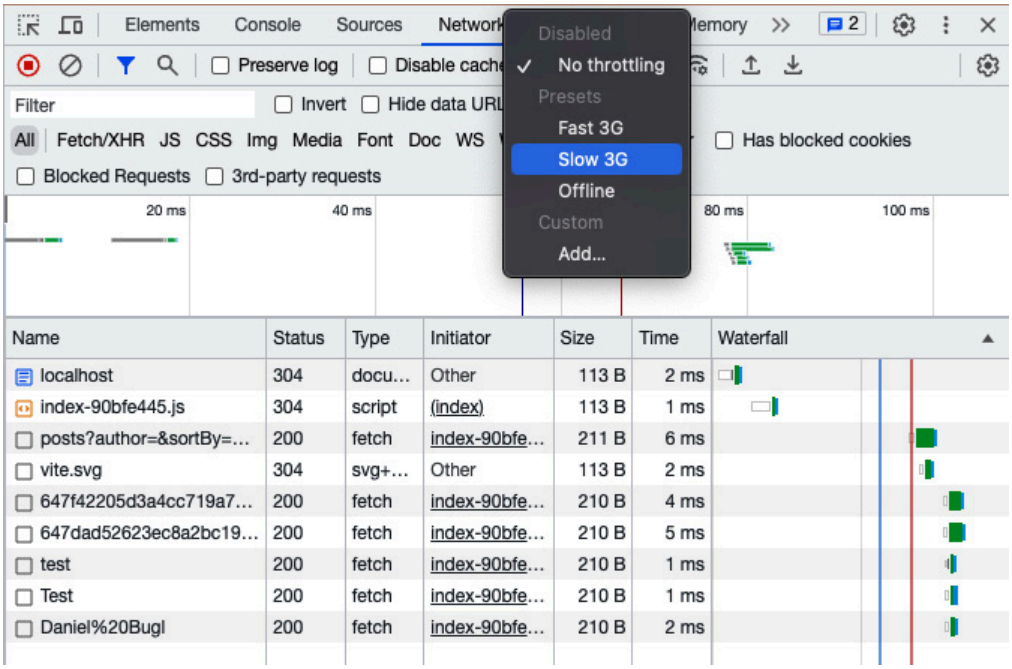

Figure 7.5 – Simulating slow networks in Google Chrome DevTools

NOTE

Lighthouse has a form of throttling built in, which is like the network throttling we are using here, but not the same. While the network throttling in DevTools is a fixed delay added to all requests, the throttling in Lighthouse attempts to simulate a more realistic scenario by adjusting the throttling based on the data observed in the initial unthrottled load.

3. Refresh the page. You will now see the app slowly loading the main layout, then a list of all posts, and finally resolving the author IDs to usernames.

This is how our page would load on slow networks. Now, the overall time to finish loading our app is almost nine seconds! You can look at the waterfall diagram to see why this is happening:

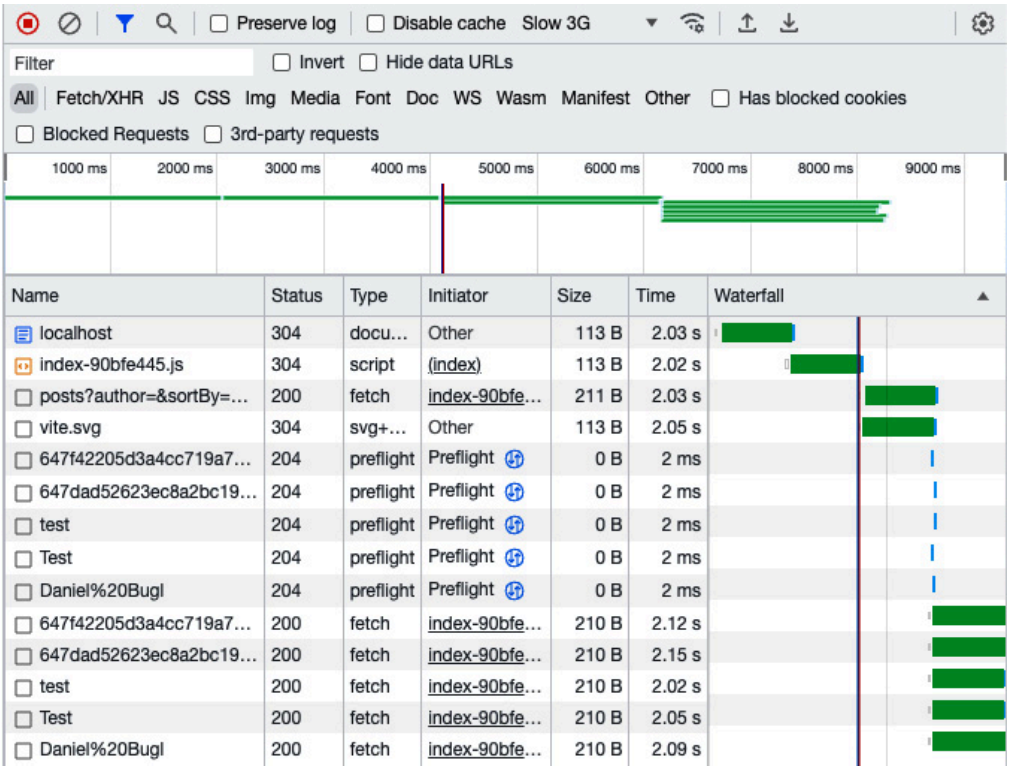

Figure 7.6 – Checking the waterfall diagram with Slow 3G throttling on

The issue in our app is that the requests are cascading. First, the HTML document loads, which then loads the JavaScript file for our app. This JavaScript file is then executed and starts rendering the layout and fetching the list of posts. After the posts are loaded, multiple requests are made in parallel to resolve the author names. As each request takes a bit over two seconds on our simulated slow network, we end up with a total load time of over eight seconds.

Now that we have learned how to benchmark a web application and found a performance bottleneck in our app (the cascading requests), let's learn how to improve the performance!

Rendering React components on the server

In the previous section, we identified cascading requests as the problem for our bad performance on slow connections. Possible solutions to this problem are as follows:

- **Bundled requests**: Fetch everything on the server and then serve everything at once to the client in a single request. This would solve the

cascading requests when fetching author names, but not the initial waiting time between the HTML page being loaded and the JavaScript executing to start fetching the data. With a latency of two seconds per request, that's still four seconds added (two seconds for loading the JavaScript and two seconds for making the request) after the HTML is fetched.

- **Server-side rendering**: Render the initial user interface with all data on the server and serve it instead of the initial HTML that just contains a URL to the JavaScript file. This would mean that no additional requests are needed to fetch the data or JavaScript and we can show the blog posts right away. Another advantage of this approach is that it allows for caching the results, so, we only need to regenerate the page on the server when a blog post gets added. A downside of this approach is that it puts more strain on the server, especially when the pages are complex to render.

In cases where data does not change so frequently or the same data is accessed by all users, server-side rendering is beneficial. In cases where data frequently changes or is personalized to each user, it might make more sense to bundle the requests into one by making a new route or using a system that can aggregate requests, such as GraphQL, which we will learn more about later in this book, in ***Chapter 11***, *Building a Backend With a GraphQL API*. In this chapter, however, we will focus on the server-side rendering approach.

Let's have a look at the differences between server-side rendering as opposed to client-side rendering:

- In **client-side rendering**, the browser downloads a minimal HTML page, which, most of the time, only contains information on where to download a JavaScript bundle, which contains all the code that will render the app.
- In **server-side rendering**, the React components are rendered on the server and served as HTML to the browser. This ensures that the app can be rendered immediately. The JavaScript bundle can be loaded later.

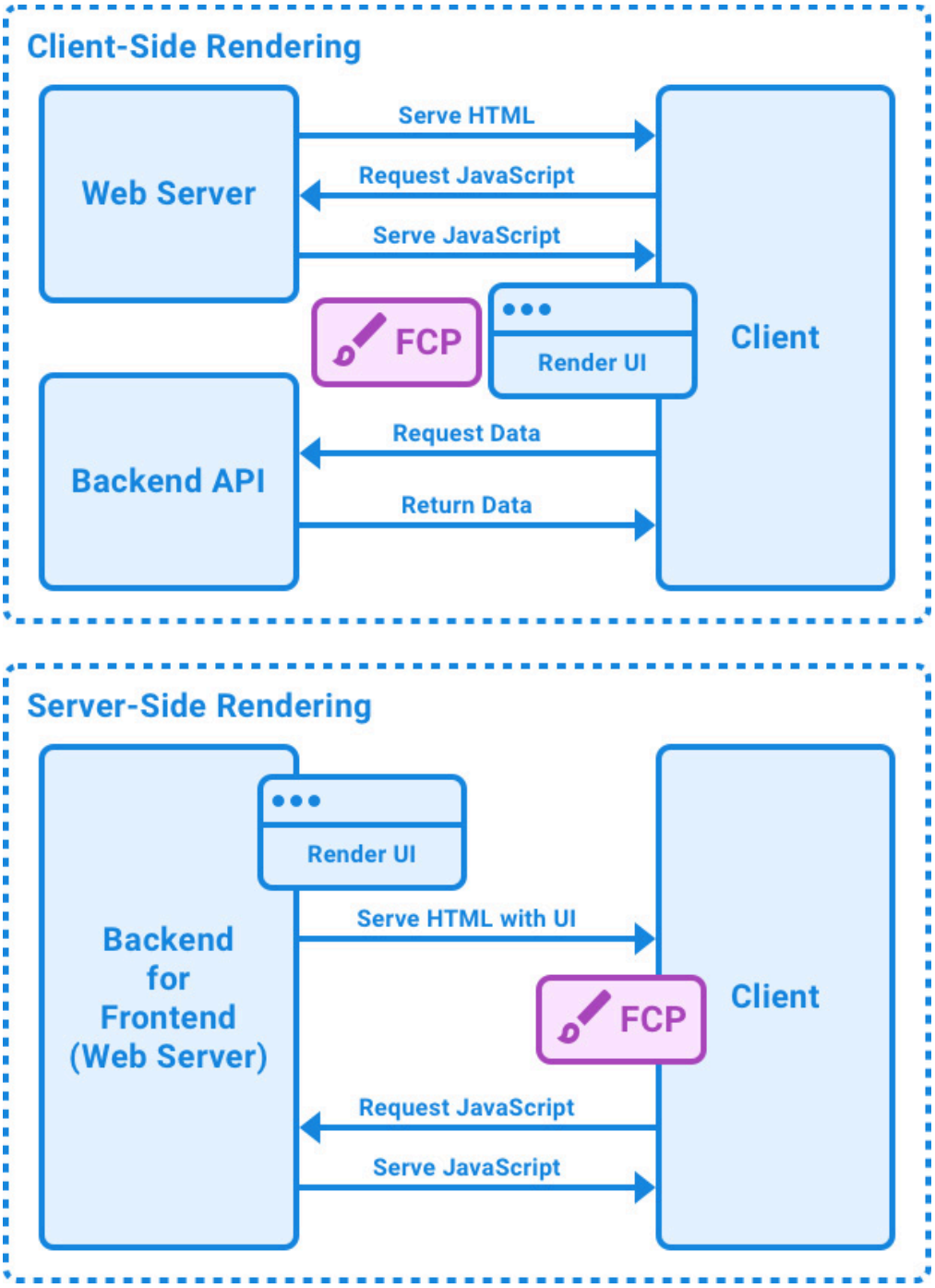

Figure 7.7 – The differences between client-side rendering and server-side rendering

It is also possible to combine the two into **isomorphic rendering**. This involves rendering the initial page on the server side, and then continuing to render changes on the client side. Isomorphic rendering combines the best of both worlds.

In addition to the performance improvements, server-side rendering is also good for **search engine optimization** (**SEO**), because search engine crawlers do not need to run JavaScript to see the page. We are going to

learn more about SEO in the next chapter, ***Chapter 8***, *Making Sure Customers Find You With Search Engine Optimization*.

Now that we have learned about server-side rendering, let's get started implementing it in our frontend, as follows:

- Setting up the server
- Defining the server-side entry point
- Defining the client-side entry point
- Updating **`index.html`** and **`package.json`**
- Making React Router work with server-side rendering

Let's start by setting up the server.

Setting up the server

Before we can get started with server-side rendering, we need to set up some boilerplate for running an Express server in tandem with Vite, so that we do not lose the benefits of Vite, such as hot reloading. Let's follow these steps to set up the server:

1. Install the **`express`** and **`dotenv`** dependencies in the root of our project (the frontend); we are going to use them to create a small web server to serve our server-side rendered page:

```
$ npm install express@4.18.2 dotenv@16.3.1
```

2. Edit **`.eslintrc.json`** and add the **`node`** env, as we are going to add server-side code to our frontend now:

```
  "env": {
    "browser": true,
    "node": true
  },
```

3. Create a new **`server.js`** file in the **`ch7`** folder, and import the **`fs`**, **`path`**, **`url`**, **`express`**, and **`dotenv`** dependencies:

```
import fs from 'fs'
import path from 'path'
import { fileURLToPath } from 'url'
```

```
import express from 'express'
import dotenv from 'dotenv'
dotenv.config()
```

4. Save the current path in a variable to be used later to reference other files in our project, using the ESM-compatible **`import.meta.url`** variable, which contains a **`file://`** URL to our project:

```
const __dirname = path.dirname(fileURLToPath(import.meta.url))
```

   We convert this URL to a regular path here.

5. Define a new **`createDevServer`** function, where we will create a Vite dev server with hot reloading and server-side rendering:

```
async function createDevServer() {
```

6. Inside this function, we first define the Express app:

```
  const app = express()
```

7. Then, import and create a Vite dev server. We use the dynamic **`import`** syntax here so that we don't need to import Vite when we define the production server later:

```
  const vite = await (
    await import('vite')
  ).createServer({
    server: { middlewareMode: true },
    appType: 'custom',
  })
  app.use(vite.middlewares)
```

   Middleware mode runs Vite as a middleware in an existing Express server. Setting **`appType`** as **`custom`** disables Vite's own serving logic so that we can control which HTML will be served.

8. Now, define a route that matches all paths and start by loading the **`index.html`** file:

```
  app.use('*', async (req, res, next) => {
    try {
      const templateHtml = fs.readFileSync(
        path.resolve(__dirname, 'index.html'),
```

```
      'utf-8',
    )
```

Make sure to load it in UTF-8 mode to support various languages and emojis in **`index.html`**.

9. Next, inject the Vite hot-module-replacement client to allow for hot reloading:

```
    const template = await vite.transformIndexHtml(
      req.originalUrl,
      templateHtml
    )
```

10. Load the entry point file for our server-side rendered app, which we will define in the next step:

```
    const { render } = await vite.ssrLoadModule('/src/entry-server.jsx')
```

The **`ssrLoadModule`** function in Vite automatically transforms the ESM source code so that it is usable in Node.js. This means we can hot-reload the entry point file without having to run a manual build.

11. Render the app using React. We will define the **`render`** function later in the server-side entry point. For now, we just call the function:

```
    const appHtml = await render()
```

12. Insert the rendered HTML from our app into the HTML template by matching a placeholder string, which we will define later in the **`index.html`** file:

```
    const html = template.replace(`<!--ssr-outlet-->`, appHtml)
```

13. Return a **`200 OK`** response with the final HTML contents:

```
    res.status(200).set({ 'Content-Type': 'text/html' }).end(html)
```

14. To wrap up the server creation, catch all errors and let Vite fix the stack trace, mapping source files in the stack trace back to the actual source code. Then, return the created Express app:

```
  } catch (e) {
    vite.ssrFixStacktrace(e)
```

```
      next(e)
    }
  })
  return app
}
```

15. Lastly, execute the **`createDevServer`** function and make the app listen on a defined port:

```
const app = await createDevServer()
app.listen(process.env.PORT, () =>
  console.log(
    `ssr dev server running on http://localhost:${process.env.PORT}`,
  ),
)
```

16. Let's not forget to define the **`PORT`** environment variable in the **`.env`** file. Edit the **`.env`** file and add the **`PORT`** environment variable, as follows:

```
VITE_BACKEND_URL="http://localhost:3001/api/v1"
PORT=5173
```

Now that we have successfully created the Express server with Vite integration, we continue by implementing the server-side entry point.

Defining the server-side entry point

The server-side entry point will use **`ReactDOMServer`** to render our React components on the server. We need to distinguish this entry point from the client-side entry point because not everything React can do is supported on the server side. Specifically, some hooks such as effect hooks will not run on the server side. Also, we will have to handle the router differently on the server side, but more on that later.

Now, let's get started defining the server-side entry point:

1. First, create a new **`src/entry-server.jsx`** file and import **`ReactDOMServer`** and the **`App`** component:

```
import ReactDOMServer from 'react-dom/server
```

```
import { App } from './App.jsx'
```

2. Define and export the render function, which returns the **`App`** component using the **`ReactDOMServer.renderToString`** function:

```
export async function render() {
  return ReactDOMServer.renderToString(
    <App />,
  )
}
```

After defining the server-side entry point, we are going to continue by defining the client-side entry point.

Defining the client-side entry point

The client-side entry point uses regular **`ReactDOM`** to render our React components. However, we need to let React know to make use of the already server-side rendered DOM. Instead of rendering, we **hydrate** the existing DOM. Like when adding water to plants, hydration makes the DOM "come alive" by adding all React functionality to the server-side rendered static DOM.

Follow these steps to define the client-side entry point:

1. Rename the existing **`src/main.jsx`** file to **`src/entry-client.jsx`**.
2. Replace the **`createRoot`** function with the **`hydrateRoot`** function, as follows:

```
ReactDOM.hydrateRoot(
  document.getElementById('root'),
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

   The **`hydrateRoot`** function accepts the component as a second argument, and does not require us to call **`.render()`**.

Now that we have defined both entry points, let's update **`index.html`** and **`package.json`**.

Updating index.html and package.json

We still need to add the placeholder string to the **`index.html`** file and adjust **`package.json`** to execute our custom server instead of the **`vite`** command directly. Let's do that now:

1. Edit **`index.html`** and add a placeholder where the server-rendered HTML will be injected:

```
<div id="root"><!--ssr-outlet--></div>
```

2. Adjust the module import to point to the client-side entry point:

```
<script type="module" src="/src/entry-client.jsx"></script>
```

3. Now, edit **`package.json`** and replace the dev script with the following:

```
"dev": "node server",
```

4. Additionally, replace the **`build`** command with commands to build the server and client:

```
"build": "npm run build:client && npm run build:server",
"build:client": "vite build --outDir dist/client",
"build:server": "vite build --outDir dist/server --ssr src/entry-server.js
```

Our setup is now ready for server-side rendering. However, when you start the server, you will immediately notice that React Router does not work with our current setup. Let's fix that now.

Making React Router work with server-side rendering

To make React Router work with server-side rendering, we need to use **`StaticRouter`** on the server side and **`BrowserRouter`** on the client side. We can reuse the same route definitions for both sides. Let's get started refactoring our code to make React Router work on the server side:

1. Edit **`src/App.jsx`** and *remove* the router-related imports (the highlighted lines) from it:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { createBrowserRouter, RouterProvider } from 'react-router-dom'
import { AuthContextProvider } from './contexts/AuthContext.jsx'
import { Blog } from './pages/Blog.jsx'
import { Signup } from './pages/Signup.jsx'
import { Login } from './pages/Login.jsx'
```

2. Import **`PropTypes`**, as we will need it later:

```
import PropTypes from 'prop-types'
```

3. Next, *remove* the following route definitions from it; we will put them in a new file soon:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Blog />,
  },
  {
    path: '/signup',
    element: <Signup />,
  },
  {
    path: '/login',
    element: <Login />,
  },
])
```

4. Adjust the function to accept **`children`** and replace **`RouterProvider`** with **`{children}`**:

```
export function App({ children }) {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthContextProvider>
        {children}
      </AuthContextProvider>
    </QueryClientProvider>
  )
}
```

5. We also need to add the **`propTypes`** definitions for the **`App`** component now:

```
App.propTypes = {
  children: PropTypes.element.isRequired,
}
```

6. Create a new **`src/routes.jsx`** file and import the previously removed imports there:

```
import { Blog } from './pages/Blog.jsx'
import { Signup } from './pages/Signup.jsx'
import { Login } from './pages/Login.jsx'
```

7. Then, add the route definitions and export them:

```
export const routes = [
  {
    path: '/',
    element: <Blog />,
  },
  {
    path: '/signup',
    element: <Signup />,
  },
  {
    path: '/login',
    element: <Login />,
  },
]
```

Now that we have refactored our app structure in a way where we can reuse the routes on the client-side and server-side entry points, let's redefine the router in the client entry point.

Defining the client-side router

Follow these steps to re-define the router in the client entry point:

1. Edit **`src/entry-client.jsx`** and import **`RouterProvider`**, the **`createBrowserRouter`** function, and **`routes`**:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { createBrowserRouter, RouterProvider } from 'react-router-dom'
import { App } from './App.jsx'
import { routes } from './routes.jsx'
```

2. Then, create a new browser router based on the **routes** definition:

```
const router = createBrowserRouter(routes)
```

3. Adjust the **render** function to render **App** with **RouterProvider**:

```
ReactDOM.hydrateRoot(
  document.getElementById('root'),
  <React.StrictMode>
    <App>
      <RouterProvider router={router} />
    </App>
  </React.StrictMode>,
)
```

Next, let's define the **server-side router**.

Mapping the Express request to a Fetch request

On the server side, we will get an Express request, which we first need to convert to a Fetch request, so that React Router can understand it. Let's do that now:

1. Create a new **src/request.js** file and define a **createFetchRequest** function there, which takes an Express request as an argument:

```
export function createFetchRequest(req) {
```

2. First, define the **origin** for the request and build the URL:

```
  const origin = `${req.protocol}://${req.get('host')}`
  const url = new URL(req.originalUrl || req.url, origin)
```

We need to use **req.originalUrl** first (if available), to take into account the Vite middleware potentially changing the URL.

3. Then, we define a new **`AbortController`** to handle when the request is closed:

```
  const controller = new AbortController()
  req.on('close', () => controller.abort())
```

4. Next, we map the Express request **`headers`** to Fetch headers:

```
  const headers = new Headers()
  for (const [key, values] of Object.entries(req.headers)) {
    if (!values) continue
    if (Array.isArray(values)) {
      for (const value of values) {
        headers.append(key, value)
      }
    } else {
      headers.set(key, values)
    }
  }
```

5. Now, we can build the **`init`** object for the Fetch request, which consists of **`method`**, **`headers`**, and **`AbortController`**:

```
  const init = {
    method: req.method,
    headers,
    signal: controller.signal,
  }
```

6. If our request was not a **`GET`** or **`HEAD`** request, we also get **`body`**, so, let's add that to the Fetch request, too:

```
  if (req.method !== 'GET' && req.method !== 'HEAD') {
    init.body = req.body
  }
```

7. Finally, let's create the Fetch **`Request`** object from our extracted information:

```
  return new Request(url.href, init)
}
```

Now that we have a utility function to convert an Express request to a Fetch request, we can make use of it to define the server-side router.

Defining the server-side router

The server-side router works very similarly to the client-side router, except that we are getting the request info from Express instead of the page, and using **`StaticRouter`**, because the route cannot change on the server side. Follow these steps to define the server-side router:

1. Edit **`src/entry-server.jsx`** and import **`StaticRouterProvider`** and the **`createStaticHandler`** and **`createStaticRouter`** functions. Also, import the **`routes`** definition and the **`createFetchRequest`** function we just defined:

```
import ReactDOMServer from 'react-dom/server'
import {
  createStaticHandler,
  createStaticRouter,
  StaticRouterProvider,
} from 'react-router-dom/server'
import { App } from './App.jsx'
import { routes } from './routes.jsx'
import { createFetchRequest } from './request.js'
```

2. Define a static handler for the routes:

```
const handler = createStaticHandler(routes)
```

3. Adjust the **`render`** function to accept an Express request object and then create a Fetch request from it using our previously defined function:

```
export async function render(req) {
  const fetchRequest = createFetchRequest(req)
```

4. We can now use this converted request to pass it to our static handler, which creates **`context`** for the route, allowing React Router to see which route we are trying to access and with which parameters:

```
  const context = await handler.query(fetchRequest)
```

5. From the routes defined by the handler and the context, we can create a static router:

```
const router = createStaticRouter(handler.dataRoutes, context)
```

6. Finally, we can adjust the rendering to render the static router and our refactored **`App`** structure:

```
  return ReactDOMServer.renderToString(
    <App>
      <StaticRouterProvider router={router} context={context} />
    </App>,
  )
}
```

7. There's still one more thing left to do. We need to pass the Express request to the **`render()`** function of the server-side entry point. Edit the following line in the **`server.js`** file:

```
    const appHtml = await render(req)
```

8. If the frontend and backend are already running, make sure to quit them.
9. Start the frontend, as follows:

```
$ npm run dev
```

10. Also, start the backend in a separate Terminal:

```
$ cd backend
$ npm run dev
```

The frontend will now output **`ssr dev server running on http://localhost:5173`** and successfully server-side render all our pages! You can verify that it is server-side rendered by opening the DevTools, clicking on the cog icon in the top right, scrolling down in the **Settings** | **Preferences** pane to the **Debugger** section, and checking the box for **Disable JavaScript**, as follows:

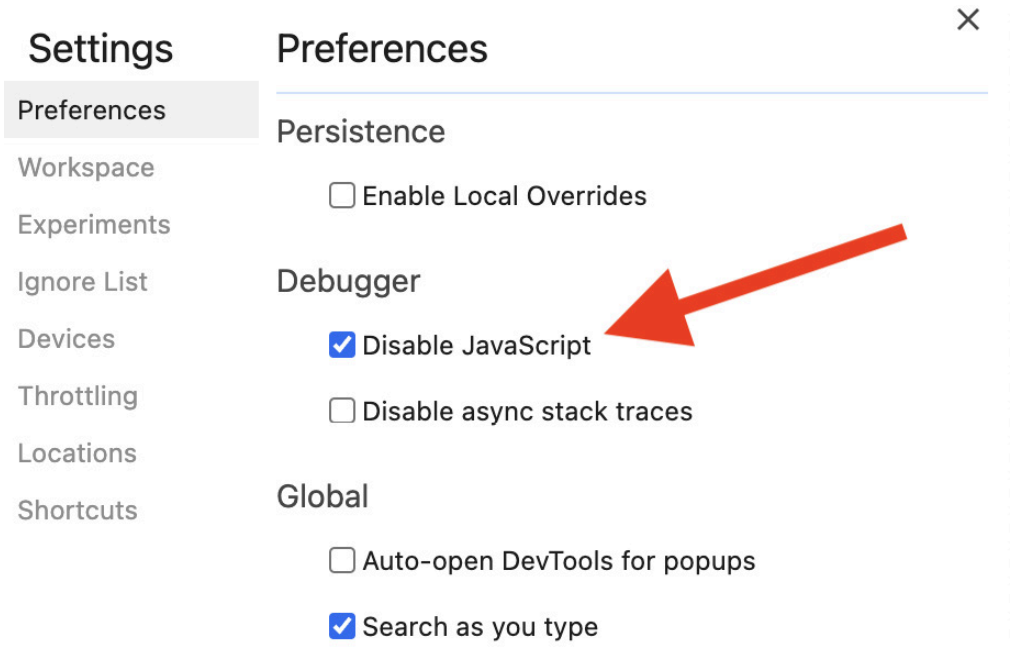

Figure 7.8 – Disabling JavaScript in the DevTools

Now, refresh the page, and you will see that part of the app still gets rendered. Only the top part of the app is fully rendered by the server side right now. The posts list is not rendered on the server side yet. This is because the **`useQuery`** hooks internally use an effect hook to fetch data after the component has mounted. As such, they do not work with server-side rendering. However, we can still get data fetching working with server-side rendering. Let's learn about that in the next section.

Server-side data fetching

As we have seen, data fetching does not work out of the box on the server side. There are two approaches for server-side data fetching with React Query:

- **Initial data approach**: Use the **`initialData`** option in the **`useQuery`** hook to pass prefetched data in. This approach is enough for fetching a list of posts but would be tricky for fetching deeply nested data, such as the usernames of each author.
- **Hydration approach**: This allows us to prefetch any requests and store the result by their query key and prefetch any request on the server side, even if it is deeply nested within the app, without having to pass the prefetched data down using props or a context.

We are first going to use the **initialData** option to fetch the list of blog posts, and then extend our solution to the hydration approach so that we can get a feeling for how both approaches work and what their pros and cons are.

Using initial data

React Router allows us to define **loaders** in routes, which we can use to fetch data on the server side and client side when the route is loaded. We can then pass the data fetched from the loaders into the **Blog** component and the **useQuery** hook via the **initialData** option. Let's do that now:

1. Edit **src/routes.jsx** and import the **useLoaderData** hook from **react-router-dom** and the **getPosts** function:

```
import { useLoaderData } from 'react-router-dom'
import { Blog } from './pages/Blog.jsx'
import { Signup } from './pages/Signup.jsx'
import { Login } from './pages/Login.jsx'
import { getPosts } from './api/posts.js'
```

2. Adjust the route to define a **loader** function, in which we simply call the **getPosts** function. We can then define a **Component()** method in which we use the **useLoaderData** hook to get the data from the loader, and pass it into the **Blog** component, as follows:

```
export const routes = [
  {
    path: '/',
    loader: getPosts,
    Component() {
      const posts = useLoaderData()
      return <Blog initialData={posts} />
    },
  },
```

3. Edit **src/pages/Blog.jsx** and import **PropTypes** there, so that we can define a new prop for the component later:

```
import PropTypes from 'prop-types'
```

4. Then, add the **initialData** prop to the **Blog** component:

```
export function Blog({ initialData }) {
```

5. Pass the **`initialData`** prop into the **`useQuery`** hook, as follows:

```
  const postsQuery = useQuery({
    queryKey: ['posts', { author, sortBy, sortOrder }],
    queryFn: () => getPosts({ author, sortBy, sortOrder }),
    initialData,
  })
```

6. Lastly, define **`propTypes`** for the **`Blog`** component:

```
Blog.propTypes = {
  initialData: PropTypes.shape(PostList.propTypes.posts),
}
```

Refresh the frontend page (with JavaScript disabled) and it will now show the post list, but without resolving the author usernames. As we can see, the initial data approach is quite simple. However, if we wanted to fetch the usernames of all authors, we would have to store them somewhere and then pass them down into the user components using either props or a context, both of which would be quite tedious and would not scale well if we need to make more requests later. Thankfully, there is another, more advanced approach, which we are going to learn about now.

Using hydration

With the hydration approach, we create a query client to prefetch any requests we want to make, and then dehydrate it, pass it to the component using a loader, and hydrate it again there. Using this approach, we can simply make any query and store it using a query key. If a component uses the same query key, it will be able to render the results on the server side. Let's implement the hydration approach now:

1. Edit **`src/routes.jsx`** and import **`QueryClient`**, the **`dehydrate`** function, and the **`Hydrate`** component from React Query:

```
import { QueryClient, dehydrate, HydrationBoundary } from '@tanstack/react-que
```

2. Also, import the **`getUserInfo`** function, as we are going to fetch user-names too now:

```
import { getUserInfo } from './api/users.js'
```

3. Adjust the loader; we are now going to create a query client there:

```
  {
    path: '/',
    loader: async () => {
      const queryClient = new QueryClient()
```

4. Then, we simulate the **`getPosts`** request from the **`Blog`** component by passing in the same default arguments to it as the component would:

```
      const author = ''
      const sortBy = 'createdAt'
      const sortOrder = 'descending'
      const posts = await getPosts({ author, sortBy, sortOrder })
```

NOTE

This duplication of default arguments is a bit problematic. However, with our current server-side rendering solution, the data fetching and component rendering are too separated to properly share the code between them. A more sophisticated server-side rendering solution, such as Next.js or Remix, can deal with this pattern better.

5. Now, we can call **`queryClient.prefetchQuery`**, with the same query key as the one that will be used by the **`useQuery`** hook in the component, to prefetch the results of the query:

```
      await queryClient.prefetchQuery({
        queryKey: ['posts', { author, sortBy, sortOrder }],
        queryFn: () => posts,
      })
```

6. Next, we use the fetched posts array to get a unique list of author IDs from them:

```
      const uniqueAuthors = posts
```

```
      .map((post) => post.author)
      .filter((value, index, array) => array.indexOf(value) === index)
```

7. We now loop through all author IDs and prefetch their information:

```
    for (const userId of uniqueAuthors) {
      await queryClient.prefetchQuery({
        queryKey: ['users', userId],
        queryFn: () => getUserInfo(userId),
      })
    }
```

8. Now that we have prefetched all the necessary data, we need to call **`dehydrate`** on **`queryClient`** to return it in a serializable format:

```
    return dehydrate(queryClient)
  },
```

9. In the **`Component()`** method, we get this dehydrated state and use the **`Hydrate`** component to hydrate it again. This hydration process makes the data accessible to the server-side rendered query client:

```
  Component() {
    const dehydratedState = useLoaderData()
    return (
      <HydrationBoundary state={dehydratedState}>
        <Blog />
      </HydrationBoundary>
    )
  },
},
```

10. Finally, we can revert the **`src/pages/Blog.jsx`** component to the previous state. We start by *removing* the **`PropTypes`** import:

```
import PropTypes from 'prop-types'
```

11. Then, we *remove* the **`initialData`** prop:

```
export function Blog({ initialData }) {
```

12. We also *remove* it in the **`useQuery`** hook:

```
  const postsQuery = useQuery({
    queryKey: ['posts', { author, sortBy, sortOrder }],
    queryFn: () => getPosts({ author, sortBy, sortOrder }),
    initialData,
  })
```

13. Lastly, we *remove* the **`propTypes`** definition:

```
Blog.propTypes = {
  initialData: PropTypes.shape(PostList.propTypes),
}
```

14. Quit the frontend via *Ctrl* + *C*, then restart it as follows:

```
$ npm run dev
```

15. Refresh the page and you will see that the full blog post list, including all author names, is properly rendered on the server side now, even with JavaScript disabled!

Let's do another benchmark to see how the performance has improved:

1. Open the Chrome DevTools.
2. Enable JavaScript again by going to the cog wheel, **Settings** | **Preferences**, and unchecking **Disable JavaScript**.
3. Go to the **Lighthouse** tab. Click on **Analyze page load** to generate a new report.

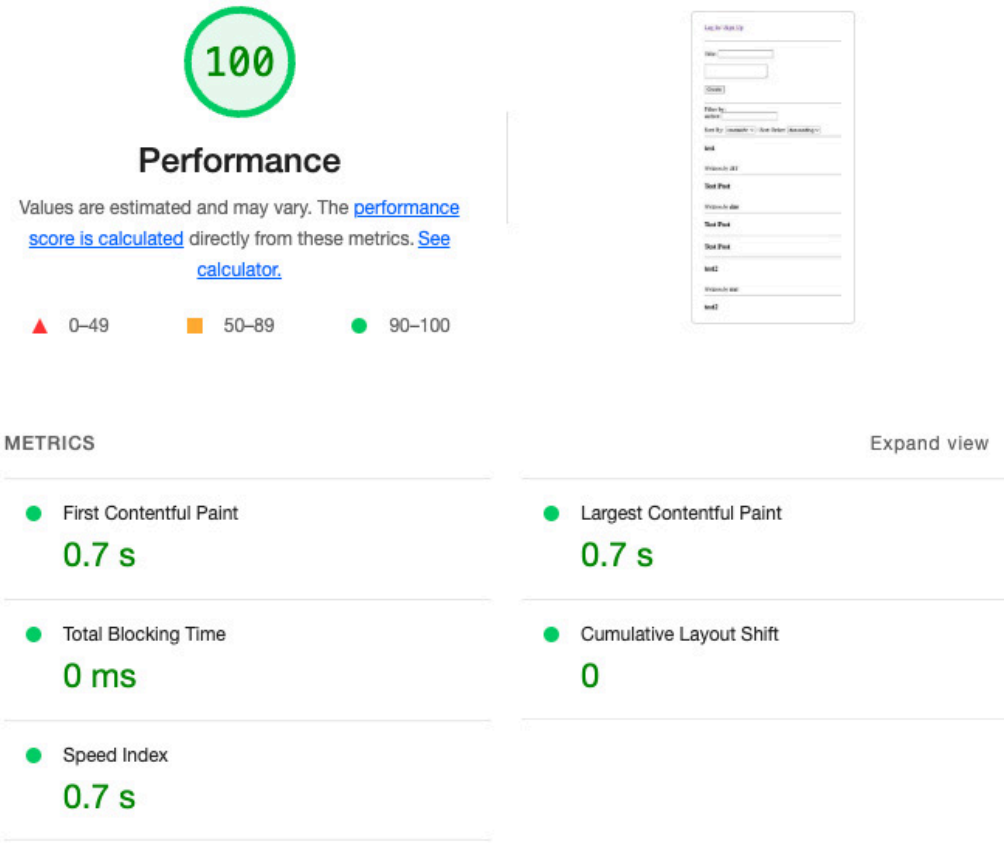

Figure 7.9 – The Lighthouse Performance score of the server-side rendered app with the dev server

The FCP and LCP times are almost half of the previously reported times from client-side rendering in production mode. Looking at the waterfall diagram in the **Network** tab, we can now see that there is only one request to fetch the initial page.

Let's now wrap up the chapter by learning about advanced server-side rendering.

Advanced server-side rendering

In the previous sections, we have successfully created a server that can do server-side rendering with hot reloading, which is very useful for development but will worsen the performance in production. Let's create another server function for a production server now, which will build files, use compression, and not load Vite middleware for hot reloading. Follow these steps to create the production server:

1. In the root of our project, install the **`compression`** dependency with the following command:

```
$ npm install compression@1.7.4
```

2. Edit **`server.js`** and define a new function for the production server, above the **`createDevServer`** function:

```
async function createProdServer() {
```

3. In this function, we define a new Express app and use the **`compression`** package and the **`serve-static`** package to serve our client:

```
  const app = express()
  app.use((await import('compression')).default())
  app.use(
    (await import('serve-static')).default(
      path.resolve(__dirname, 'dist/client'),
      {
        index: false,
      },
    ),
  )
```

4. Then, we define a route that catches all paths again, this time loading the template from the built files in the **`dist/`** folder:

```
  app.use('*', async (req, res, next) => {
    try {
      let template = fs.readFileSync(
        path.resolve(__dirname, 'dist/client/index.html'),
        'utf-8',
      )
```

5. We also directly import and render the server-side entry point now:

```
      const render = (await import('./dist/server/entry-server.js')).render
```

6. As before, we render the React app, replace the placeholder in **`index.html`** with the rendered app, and return the resulting HTML:

```
      const appHtml = await render(req)
```

```
      const html = template.replace(`<!--ssr-outlet-->`, appHtml)
      res.status(200).set({ 'Content-Type': 'text/html' }).end(html)
```

7. For the error handling, we simply pass it on to the next middleware now and return the app:

```
    } catch (e) {
      next(e)
    }
  })
  return app
}
```

8. At the bottom of the **`server.js`** file, where we created the dev server, we now do a check for the **`NODE_ENV`** environment variable and use it to decide whether to start the production server or the development server:

```
if (process.env.NODE_ENV === 'production') {
  const app = await createProdServer()
  app.listen(process.env.PORT, () =>
    console.log(
      `ssr production server running on http://localhost:${process.env.PORT}`,
    ),
  )
} else {
  const app = await createDevServer()
  app.listen(process.env.PORT, () =>
    console.log(
      `ssr dev server running on http://localhost:${process.env.PORT}`,
    ),
  )
}
```

9. Install the **`cross-env`** package, as follows:

```
$ npm install cross-env@7.0.3
```

10. Edit **`package.json`** and add a **`start`** script, which starts the server in production mode:

```
    "start": "cross-env NODE_ENV=production node server",
```

11. Quit the frontend dev server, build, and start the production server:

```
$ npm run build
$ npm start
```

As we can see, our server still serves the app just fine, but now we are not in development mode anymore, so we do not have hot reloading available. This wraps up our implementation of server-side rendering! As you can imagine, the server-side rendering implementation in this chapter is somewhat basic, and there are still multiple things we would need to handle:

- Redirects and proper HTTP status codes
- Static-site generation (caching resulting HTML pages so we don't have to server-side render them again every time)
- Better data fetching functionality
- Better code splitting between the server and client
- Better handling of environment variables between the server and client

To solve these issues, it is better to use a fully-fledged server-side rendering implementation in a web framework, such as Next.js or Remix. These frameworks already provide ways to do server-side rendering, data fetching, and routing out of the box, and do not require us to manually get everything to work in tandem. We are going to learn more about Next.js in ***Chapter 16***, *Getting Started with Next.js*.

Summary

In this chapter, we first learned how to benchmark web applications using Lighthouse and Chrome DevTools. We also learned about useful metrics for such benchmarks, called Core Web Vitals. Then, we learned about rendering React components on the server and the differences between client-side rendering and server-side rendering. Next, we implemented server-side rendering for our app using Vite and React Router. Then, we implemented server-side data fetching using React Query. We then benchmarked our app again and saw an improvement in the performance of more than 40%. Lastly, we learned about getting our server-side render-

ing server ready for production and concepts that a more sophisticated server-side rendering framework needs to deal with.

In the next chapter, ***Chapter 8***, *Making Sure Customers Find You with Search Engine Optimization*, we are going to learn how to make our web app more accessible to search engine crawlers, increasing the SEO score that we saw in the Lighthouse report. We are going to add meta tags to have more information about our web app and add integrations for various social media sites.